Name of the Student	**KOTHAKOTA RAKESH**
Reg. No	**192525075**
GitHub Link	**https://github.com/Rakesh192525075/MLA0403.git**

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 1

Case Study

COURSE NAME: DEEP LEARNING **COURSE CODE: MLA04**
SLOT: A

COURSE OUTCOME COVERED

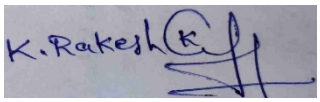
CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%
Assessment Tool Weightage: 25%

Duration: 90 minutes
Marks: 25

Code of Conduct:

I Kothakota Rakesh / 192525075 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student **Name of the student : K.Rakesh**

Criteria	**Conceptual Understanding (2.5)**	Linkages **(2.5)**	Organization **(2)**	Integration of Literature **(2)**	**Clarity & Presentation *(1)***	**Total (10)**
Weightage	***25 %***	***25%***	***20%***	***20 %***	***10 %***	**100%**
Q1						
Q2						
Total						

Signature of the Faculty: ***Total Marks :***

QUESTIONS: 02 **Total Marks: 20**

Case Study Question 1: CNN for Automated Medical Image Classification (10 Marks)
A healthcare organization wants to develop a deep learning system to automatically classify chest X-ray images into **normal** and **disease-affected** categories. The dataset contains thousands of X-ray images with variations in size, brightness and image quality. A CNN model is proposed because it can automatically learn important visual features such as edges, textures, and abnormal regions without manually designing features.

Question:

1. Design and explain a suitable **CNN architecture** for this medical image classification system.
2. Describe the **motivation for using CNNs** and explain the role of **convolutional layers, filters, pooling layers, and fully connected layers** in the proposed architecture.
3. Explain how **parameter sharing** reduces the number of trainable parameters compared with a fully connected network.
4. Discuss suitable **regularization techniques such as dropout, data augmentation and batch normalization** to reduce overfitting.
5. Finally, explain whether **AlexNet or ResNet** would be more appropriate for this application and justify your choice based on feature learning, network depth, computational requirements and classification performance.

1. Proposed CNN Architecture

For classifying chest X-ray images into normal and disease-affected categories, a CNN architecture inspired by proven designs (VGG-style blocks with residual connections) is proposed. The pipeline is as follows:

- Input Layer: Grayscale/RGB X-ray images resized and normalized to a fixed size (e.g., 224x224x3), with pixel values scaled to [0,1].
- Convolutional Block 1: 2 Conv layers (32 filters, 3x3 kernel, ReLU activation, same padding) + Batch Normalization + Max Pooling (2x2).
- Convolutional Block 2: 2 Conv layers (64 filters, 3x3) + BatchNorm + Max Pooling.
- Convolutional Block 3: 2 Conv layers (128 filters, 3x3) + BatchNorm + Max Pooling.
- Convolutional Block 4: 2 Conv layers (256 filters, 3x3) + BatchNorm + Max Pooling (deeper blocks to capture abstract disease-related patterns such as opacities and nodules).
- Global Average Pooling layer to reduce spatial dimensions and control overfitting.
- Fully Connected Layer(s): One dense layer (128-256 neurons, ReLU) followed by Dropout.
- Output Layer: A single neuron with Sigmoid activation for binary classification (normal vs disease-affected), or Softmax if extended to multi-class.

2. Motivation for Using CNNs

Traditional machine learning approaches to medical imaging require handcrafted features (edge detectors, texture descriptors) designed by domain experts, which is time-consuming and often fails to capture subtle disease markers. CNNs automatically learn hierarchical visual features directly from raw pixel data through end-to-end training. Early layers learn low-level features (edges, gradients, contrast changes), middle layers combine these into textures and shapes (rib structures, lung boundaries), and deeper layers learn high-level, disease-specific patterns (opacities, consolidations, nodules). This automatic feature

learning makes CNNs highly effective and adaptable for the variability seen in X-ray datasets, including differences in brightness, contrast, and image quality.

Role of Key Layers

- Convolutional Layers: Apply learnable filters (kernels) that slide across the image to detect local patterns such as edges and textures. Each filter produces a feature map highlighting where a specific pattern occurs.
- Filters: Small weight matrices (e.g., 3x3) that are convolved with the input; different filters specialize in detecting different visual cues (vertical edges, blobs, textures indicative of infection).
- Pooling Layers (Max Pooling): Downsample feature maps, reducing spatial dimensions and computation while retaining the most salient activations, and providing a degree of translation invariance so that a shifted abnormality is still detected.
- Fully Connected Layers: Take the flattened, high-level features from the convolutional base and learn the non-linear combinations needed to make the final classification decision.

3. Parameter Sharing and Reduced Trainable Parameters

In a fully connected network, every input pixel is connected to every neuron in the next layer, so the number of weights grows enormously with image size (e.g., a 224x224x3 image connected to just 100 neurons would require over 15 million weights in a single layer). In a CNN, a single filter (e.g., 3x3x3 = 27 weights plus a bias) is slid across the entire image and reused at every spatial location. This weight sharing means:

- The same filter detects a given pattern (e.g., an edge) regardless of where it appears in the image, which is appropriate since a disease marker could appear anywhere in the lung field.
- The number of trainable parameters depends only on the filter size and number of filters, not on the image dimensions, drastically reducing the parameter count compared to a fully connected layer.
- Fewer parameters lower memory and computational requirements and reduce the risk of overfitting, which is critical when medical datasets are relatively small compared to natural image datasets.

4. Regularization Techniques to Reduce Overfitting

- Dropout: Randomly deactivates a fraction of neurons (e.g., 30-50%) during training, preventing the network from becoming overly reliant on specific neurons and forcing it to learn redundant, generalizable representations.
- Data Augmentation: Since medical datasets are often limited, applying rotations, flips, zoom, brightness/contrast adjustments, and slight translations artificially expands the training set and helps the model generalize to real-world variations in imaging conditions.
- Batch Normalization: Normalizes the activations within each mini-batch, stabilizing and accelerating training, allowing higher learning rates, and providing a mild regularization effect that reduces the need for aggressive dropout.
- Additional measures: Early stopping based on validation loss, L2 weight regularization, and using a held-out validation set to monitor generalization can further reduce overfitting.

5. AlexNet vs. ResNet — Recommendation

Between AlexNet and ResNet, ResNet is the more appropriate choice for this medical image classification application, for the following reasons:

- Feature Learning: ResNet's residual (skip) connections allow the network to learn much richer, more discriminative hierarchical features, which is valuable for detecting subtle abnormalities in X-rays.
- Network Depth: ResNet can be trained effectively at much greater depths (e.g., 50, 101 layers) because residual connections mitigate the vanishing gradient problem, whereas AlexNet is a shallow 8-layer network with limited representational capacity.
- Computational Requirements: While ResNet is deeper, techniques like bottleneck blocks keep parameter counts efficient; AlexNet, despite being shallower, has large fully connected layers that make it parameter-heavy without matching accuracy.
- Classification Performance: ResNet architectures consistently outperform AlexNet on complex image classification benchmarks and are widely adopted (often via transfer learning) in medical imaging applications due to their superior accuracy and robustness.

Overall, ResNet's ability to train deeper networks without degradation, combined with transfer learning from pretrained ImageNet weights, makes it better suited to capture the fine-grained patterns needed for reliable disease detection in chest X-rays.

Case Study Question 2: CNN-Based Autonomous Vehicle Object Recognition (10 Marks)
An autonomous vehicle company is developing a vision system that must identify **cars, pedestrians, traffic signs, bicycles, and road obstacles** from images captured by cameras mounted on the vehicle. The system must recognize objects under different lighting conditions, viewing angles, distances and road environments. The development team is considering **AlexNet and ResNet** architectures for building the CNN-based recognition system.
Question:

1. Analyze how a **CNN architecture** can be designed for this autonomous vehicle application.
2. Explain the purpose of the **input, convolution, activation, pooling, and fully connected/output layers** and describe how convolutional **filters progressively learn low-level and high-level features**.
3. Explain the importance of **parameter sharing** in reducing computational complexity and improving translation-related feature detection.
4. Discuss the possible problem of **overfitting** and recommend suitable regularization methods.
5. Compare **AlexNet and ResNet** in terms of architecture, depth, parameter efficiency, training difficulty, vanishing-gradient problems and feature-learning capability.
6. Finally, recommend the most suitable architecture for the autonomous vehicle system and justify your decision based on **accuracy, computational cost and real-time application requirements**.

1. CNN Architecture Design for Object Recognition

For detecting and classifying cars, pedestrians, traffic signs, bicycles, and obstacles under varying lighting, angles, and distances, a deep CNN architecture (such as a ResNet-based backbone combined with detection heads, e.g., similar to Faster R-CNN or YOLO-style detectors) is proposed:

- Input Layer: Camera-captured RGB images, resized and normalized (e.g., 416x416x3 or 224x224x3 depending on the detection framework).

- Convolutional Layers: A stack of convolutional layers with increasing filter counts (e.g., 32, 64, 128, 256, 512) to progressively extract spatial features.
- Activation Layers: ReLU (or Leaky ReLU) applied after each convolution to introduce non-linearity, enabling the network to learn complex decision boundaries needed to distinguish visually similar classes (e.g., cyclists vs. pedestrians).
- Pooling Layers: Max pooling / strided convolutions reduce spatial resolution, control computation, and build translation invariance so objects are recognized regardless of their exact pixel position.
- Fully Connected / Output Layers: Dense layers (in classification-only settings) or detection heads (bounding box regression + class probabilities in an object-detection setting) produce the final predictions — object class and location.

How Convolutional Filters Learn Low- to High-Level Features

Early convolutional layers learn simple, low-level features such as edges, corners, and color gradients. As the network deepens, filters combine these low-level features into mid-level patterns such as wheel shapes, contours of pedestrians, or the geometric shapes of traffic signs. The deepest layers assemble these into high-level, semantically meaningful representations — a complete car, a pedestrian's silhouette, or the specific shape and color pattern of a stop sign — enabling robust recognition across varied lighting, angles, and distances.

2. Importance of Parameter Sharing

- Computational Efficiency: Reusing the same filter weights across all spatial locations drastically reduces the number of parameters compared to a fully connected approach, making it feasible to process high-resolution video frames in real time on onboard vehicle hardware.
- Translation-Related Feature Detection: Since a filter is applied identically across the image, an object (e.g., a pedestrian) is detected consistently whether it appears on the left, right, or center of the frame — essential for a moving vehicle where objects constantly shift position within the camera view.
- Faster Training and Lower Memory Footprint: Fewer unique parameters mean faster convergence and reduced memory/storage requirements, which is important for embedded, real-time automotive systems with strict latency and power constraints.

3. Overfitting and Regularization

Because road scenes vary enormously (weather, lighting, occlusion, road type), a model trained on limited driving data risks overfitting to specific conditions and failing to generalize. Recommended regularization methods include:

- Data Augmentation: Simulating varied lighting, weather, occlusion, rotation, scaling, and cropping to expose the model to diverse driving conditions.
- Dropout: Applied in fully connected/detection head layers to prevent reliance on specific feature combinations.
- Batch Normalization: Stabilizes training across varying batch statistics caused by diverse scene conditions and provides mild regularization.
- Transfer Learning and Large-Scale Pretraining: Using models pretrained on large driving datasets (e.g., BDD100K, KITTI) improves generalization before fine-tuning on company-specific data.
- Early Stopping and Cross-Validation: Monitoring performance on a diverse validation set to prevent the model from overfitting to the training distribution.

4. AlexNet vs. ResNet — Comparison

Criterion	AlexNet	ResNet
Architecture	8 layers (5 conv + 3 FC), simple stacked design	Deep architecture built from residual blocks with skip/identity connections
Depth	Shallow (8 layers)	Very deep (18 to 152+ layers)
Parameter Efficiency	Large FC layers make it parameter-heavy (~60M parameters)	Bottleneck blocks and global average pooling keep parameters efficient relative to depth
Training Difficulty	Relatively easier to train due to shallow depth, but limited capacity	Deeper networks are harder to train, but residual connections make optimization tractable
Vanishing Gradient	More susceptible if depth increases, limiting scalability	Skip connections allow gradients to flow directly, effectively solving vanishing gradients
Feature-Learning Capability	Limited hierarchical feature abstraction due to shallow depth	Rich, multi-level feature representations due to depth, better suited for complex, cluttered scenes

5. Recommendation for the Autonomous Vehicle System

ResNet is the recommended architecture for this autonomous vehicle object recognition system:

- Accuracy: ResNet's depth and residual learning enable it to accurately distinguish visually similar and small/partially occluded objects (e.g., pedestrians partially hidden behind vehicles), which is critical for safety.
- Computational Cost: Although deeper, ResNet's bottleneck design and efficient use of parameters make it feasible to deploy (often in compressed/optimized forms such as ResNet-18/34 or via pruning and quantization) on automotive-grade hardware.
- Real-Time Requirements: With hardware acceleration (GPUs/TPUs on-board) and optimized variants, ResNet backbones are widely used in production-grade real-time detection systems (e.g., as backbones in Faster R-CNN, RetinaNet), striking a strong balance between speed and accuracy.

Overall, given the safety-critical nature of autonomous driving and the need to detect diverse object classes reliably across varying real-world conditions, ResNet's superior feature learning and robustness to vanishing gradients outweigh AlexNet's simplicity, making it the more suitable choice.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand